

Nome: André Machado Silva

Repositório no GitHub:

<https://github.com/andrems1/onboarding-LIPAI>

Código das Videoaulas

```
import pandas as pd

s = pd.Series([10, 20, 30, 40], index=['a', 'b', 'c', 'd'])
print(s)
print(s['b'])
print(s[s > 15])

data = {'nome': ['Ana', 'Bruno', 'Carla'],
        'idade': [25, 30, 22],
        'nota': [8.5, 7.0, 9.2]}
df = pd.DataFrame(data)
print(df.head())
print(df.info())
print(df.describe())

print(df['nome'])
print(df[['nome', 'nota']])
print(df.iloc[0])
print(df.loc[df['nota'] > 8])

df['aprovado'] = df['nota'] >= 7.0
df['nota_pct'] = df['nota'] / 10 * 100

print(df.groupby('aprovado')['nota'].mean())
print(df['nota'].agg(['mean', 'std', 'min', 'max']))
print(df.sort_values('nota', ascending=False))
```

Exercícios

ex01

```
import pandas as pd

df = pd.read_csv('classification_results_trial_0001.csv')

print(df.head())
df.info()
print(df.describe())
```

1. Quantas imagens são benign e malign em real_class?

```
print(df['real_class'].value_counts())
```

Resultado: malign = 55, benign = 45

2. Imagens onde o modelo errou a predição

```
df['acerto'] = df['real_class'] == df['predicted_class']
erros = df[~df['acerto']]
print(erros[['image_path', 'real_class', 'predicted_class', 'prob_benign', 'prob_malign']])
```

Resultado: 12 imagens com predição errada.

3. Confiança do modelo nos erros

```
erros = erros.copy()
erros['prob_predita'] = erros.apply(
    lambda r: r['prob_malign'] if r['predicted_class']=='malign'
    else r['prob_benign'], axis=1)
confiantes = erros[erros['prob_predita'] >= 0.7]
print(confiantes[['image_path', 'real_class', 'predicted_class', 'prob_predita']])
```

Resultado: 7 erros com confiança >= 70%.

4. Calcular TP, TN, FP, FN

```
TP = len(df[(df['real_class']=='malign') & (df['predicted_class']=='malign')])
TN = len(df[(df['real_class']=='benign') & (df['predicted_class']=='benign')])
FP = len(df[(df['real_class']=='benign') & (df['predicted_class']=='malign')])
FN = len(df[(df['real_class']=='malign') & (df['predicted_class']=='benign')])
print(f'TP={TP} TN={TN} FP={FP} FN={FN}')
```

Resultado: TP=46, TN=42, FP=3, FN=9

5. Métricas

```
accuracy = (TP+TN)/(TP+TN+FP+FN)
precision = TP/(TP+FP)
recall = TP/(TP+FN)
specificity = TN/(TN+FP)
print(f'Acurácia: {accuracy:.4f}')
```

```
print(f'Precisão: {precision:.4f}')
```

```
print(f'Recall: {recall:.4f}')
```

```
print(f'Especificidade: {specificity:.4f}')
```

Resultado: Acurácia=0.8800, Precisão=0.9388, Recall=0.8364, Especificidade=0.9333

6. 5 imagens benign com menor prob_benign

```
benign_df = df[df['real_class']=='benign'].nsmallest(5, 'prob_benign')
print(benign_df[['image_path', 'prob_benign', 'prob_malign', 'predicted_class']])
```

```
image_068.jpg prob_benign=0.1865 previsto=malign
image_042.jpg prob_benign=0.2518 previsto=malign
image_081.jpg prob_benign=0.3411 previsto=malign
image_048.jpg prob_benign=0.5027 previsto=benign
image_004.jpg prob_benign=0.5086 previsto=benign
```

Essas imagens têm características atípicas para benigno; o modelo ficou inseguro ou errou.

7. 5 imagens malign com maior prob_benign

```
malign_df = df[df['real_class']=='malign'].nlargest(5, 'prob_benign')
print(malign_df[['image_path', 'prob_benign', 'prob_malign', 'predicted_class']])

image_092.jpg prob_benign=0.8972 previsto=benign
image_028.jpg prob_benign=0.8607 previsto=benign
image_066.jpg prob_benign=0.8353 previsto=benign
image_027.jpg prob_benign=0.8180 previsto=benign
image_021.jpg prob_benign=0.8074 previsto=benign
```

O modelo subestimou a malignidade dessas imagens — casos críticos de falso negativo.